Fruit Classification

Data Science Project Documentation

Zaid Shawakfeh

Computer Engineering — German Jordanian University

March 2026

Supervisor: Gustav Willig

Table of Contents

1. Problem Statement 3

2. Data Preprocessing 4

2.1 Step-by-Step Preprocessing 4

Step 1 — Removing Missing Values 4

Step 2 — Lowercasing and Stripping Whitespace 4

Step 3 — Removing Stray Digits from Text Columns 4

Step 4 — Fuzzy Matching for Typo Correction 4

Step 5 — Per-Fruit Outlier Removal (IQR Method) 5

Step 6 — Data Type Enforcement and Renaming 5

2.2 Feature Encoding for Model Input 5

3. Correlation Analysis 6

4. Model Overview 7

4.1 Decision Tree 7

4.2 Random Forest 7

4.3 Logistic Regression 8

5. Hyperparameter Tuning: GridSearchCV + K-Fold 9

5.1 Stratified K-Fold Cross Validation 9

5.2 GridSearchCV 9

5.3 Overfitting Detection 9

6. Model Results 10

6.1 Decision Tree 10

6.2 Random Forest 10

6.3 Logistic Regression 11

7. Model Comparison 12

7.1 Overall Performance 12

7.2 Per-Class Analysis 12

7.3 Recommendation 13

8. Advantages and Disadvantages of Each Model 14

8.1 Decision Tree 14

8.2 Random Forest 14

8.3 Logistic Regression 14

9. Future Work **15**

10. Conclusion **16**

1. Problem Statement

The goal of this project is to build a classifier that can tell apart three types of fruit: apples, bananas, and grapes. The dataset has 200 records with four columns: fruit type, color, size, and weight. The fruit type is what we want to predict, and the other three columns are the features we use to make that prediction.

I chose Decision Tree and Logistic Regression as my two main models because they are interpretable — meaning you can actually understand why the model made a certain decision, which I think is important especially when presenting results to someone else. I also added Random Forest as an extra model because I was curious whether combining many trees together would give better results than a single tree, and I wanted to go a step further than the minimum requirements.

All models were tuned using GridSearchCV with Stratified K-Fold Cross Validation, and evaluated using accuracy, F1 score, ROC-AUC, and a full classification report per class. The entire project was built in Python inside a Jupyter Notebook.

2. Data Preprocessing

Before training any model, I had to clean the dataset quite a bit. Looking at the raw data, I noticed several issues: some entries had typos like 'largee' instead of 'large', some color values had numbers attached to them like 'Yellow1', and the weight column had outlier values that were clearly wrong. I handled all of this in a single preprocessing cell to keep everything traceable and easy to follow.

2.1 Step-by-Step Preprocessing

Step 1 — Removing Missing Values

The first thing I did was remove any row that had a missing value in any column. This is a basic but important step — incomplete records could cause errors later or quietly skew the model's learning.

```
df.dropna(inplace=True)
```

Step 2 — Lowercasing and Stripping Whitespace

I converted all text columns to lowercase and removed extra spaces. Without this, 'Yellow' and 'yellow' would be treated as two different colors, which doesn't make sense.

```
df[col] = df[col].str.strip().str.lower()
```

Step 3 — Removing Stray Digits from Text Columns

Some entries had random numbers mixed in with the text, like 'Yellow1'. I used a regular expression to strip all digits from the color and size columns, which fixed these cases automatically.

```
df[col] = df[col].apply(lambda x: re.sub(r"\d+", "", str(x)).strip())
```

Step 4 — Fuzzy Matching for Typo Correction

Even after removing digits, there were still typos like 'largee'. I used Python's difflib.get_close_matches() to compare each value against a list of valid options and replace it

with the closest match. I set the similarity threshold at 75%, which was enough to catch the typos without accidentally changing correct values.

```
matches = get_close_matches(value, valid_list, n=1, cutoff=0.75)
```

Step 5 — Per-Fruit Outlier Removal (IQR Method)

For the weight column, I removed outliers using the IQR method. The important thing here is that I computed the IQR separately for each fruit type, not across the whole dataset. This matters because a weight that is normal for an apple would be a clear outlier for a grape. This step removed 7 rows, leaving 193 clean records.

```
Q1  = df.groupby('fruit_type')['weight'].transform('quantile', 0.25)
Q3  = df.groupby('fruit_type')['weight'].transform('quantile', 0.75)
IQR = Q3 - Q1
mask = (df['weight'] >= Q1 - 1.5*IQR) & (df['weight'] <= Q3 + 1.5*IQR)
```

Step 6 — Data Type Enforcement and Column Renaming

After cleaning, I made sure the text columns were stored as plain strings, and I renamed the fruit_type column to 'label' since that is the target output class.

```
df[col] = df[col].astype(str)
df.rename(columns={"fruit_type": "label"}, inplace=True)
```

2.2 Feature Encoding for Model Input

Machine learning models only work with numbers, so I encoded the color and size columns using LabelEncoder from scikit-learn. The final set of input features for all models was: color_enc, size_enc, and weight.

```
df_model["color_enc"] = le_color.fit_transform(df_model["color"])
df_model["size_enc"]  = le_size.fit_transform(df_model["size"])
FEATURES = ["color_enc", "size_enc", "weight"]
```

3. Correlation Analysis

Before jumping into model building, it is useful to look at how the features relate to each other and to the target label. Correlation analysis helps answer a simple but important question: which features actually carry useful information for predicting the fruit type?

Looking at the results across all three models, weight turned out to be by far the most important feature. Grapes are significantly lighter than apples and bananas, which is why all three models — even Logistic Regression — managed to classify grapes well. Weight alone created a natural and clean separation for that class.

Color and size, on the other hand, were less helpful. Apples and bananas share overlapping size categories and can have similar colors like green or yellow, which made it much harder for the models to tell them apart. This overlap is the main reason why apple and banana F1 scores were consistently lower than grape F1 scores across all models.

A formal correlation heatmap between the encoded features confirms this — weight has the strongest relationship with the label, while color_enc and size_enc contribute less discriminative power. This finding suggests that adding new features in future work, such as fruit shape or texture, could significantly improve the classification of apples and bananas.

4. Model Overview

4.1 Decision Tree

I chose the Decision Tree as one of my main models because it is easy to understand and interpret. The way it works is straightforward — it asks a series of yes/no questions about the features and follows the answers down a tree until it reaches a prediction. This kind of transparency is valuable because you can actually explain to someone why the model classified a fruit as an apple rather than a banana.

The main risk with Decision Trees is overfitting. If you let the tree grow too deep, it starts memorising the training data instead of learning general patterns. That is why I used GridSearchCV to find the right max_depth and other regularization parameters.

Key hyperparameters tuned:

- max_depth — controls how deep the tree can grow. Too deep = overfitting, too shallow = underfitting.
- criterion — the splitting metric: gini or entropy (information gain).
- min_samples_split — minimum samples needed to split a node.
- min_samples_leaf — minimum samples required at a leaf. Acts as a regularizer.

4.2 Random Forest

I added Random Forest because I was curious whether using many trees together would give better results than a single tree. The idea behind it is that each tree is trained on a slightly different random sample of the data, and the final prediction is a majority vote across all trees. This makes the model much more stable and less sensitive to noise in the training data.

Why Random Forest is Stronger Than a Single Decision Tree

A single Decision Tree is fragile — a small change in the training data can completely change the tree structure. Random Forest fixes this by averaging hundreds of trees, each of which sees a different slice of the data and features. Their individual errors cancel out, and the result is a more reliable prediction. It also provides a feature importance score, which showed that weight was the most useful feature by a large margin in this dataset.

Key hyperparameters tuned:

- n_estimators — number of trees. More trees means more stability but slower training.
- max_depth — maximum depth per tree.
- max_features — number of features considered at each split. 'sqrt' is the standard for classification.
- min_samples_split / min_samples_leaf — same regularization role as in a single Decision Tree.

4.3 Logistic Regression

Logistic Regression was included as the second required model. Unlike the tree models, it tries to find a straight line (or flat surface in higher dimensions) that separates the classes. It is fast and interpretable, and you can directly read the model's coefficients to understand how each feature influences the prediction.

The limitation here is that it only works well when the classes are linearly separable, meaning they can be divided by a straight boundary. As the results showed, that is not really the case with this

dataset — especially for apples and bananas, which overlap significantly in the feature space. Because of this, Logistic Regression ended up performing noticeably worse than both tree-based models.

Since Logistic Regression is sensitive to feature scale, I wrapped it inside a scikit-learn Pipeline with StandardScaler to make sure the scaler was only fitted on the training data and never leaked information from the test set.

Key hyperparameters tuned:

- C — inverse regularization strength. Small C = stronger regularization = simpler model.
- penalty — type of regularization. L2 (Ridge) was used throughout.
- solver — optimization algorithm. Both lbfgs and saga were evaluated.
- multi_class='multinomial' — enables proper multi-class probability estimation for all three fruits.

5. Hyperparameter Tuning: GridSearchCV + K-Fold Cross Validation

5.1 Stratified K-Fold Cross Validation

Instead of just splitting the data once into train and test, I used K-Fold Cross Validation to get a more reliable estimate of how each model performs. The idea is to split the training data into 5 equal parts (folds), train the model 5 times — each time leaving a different fold out as the validation set — and then average the results. This gives a much better picture of how the model generalises than a single split.

I specifically used Stratified K-Fold, which makes sure each fold has the same proportion of apples, bananas, and grapes as the original dataset. Without this, one fold might end up with too many grapes and too few apples, which would make the validation results misleading.

```
kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

5.2 GridSearchCV

GridSearchCV automates hyperparameter tuning by trying every possible combination of parameter values I defined in a grid. For each combination, it runs the K-Fold cross validation and picks the combination with the best average F1 weighted score. This is much more reliable than manually guessing hyperparameter values one by one.

By passing the StratifiedKFold object directly to GridSearchCV, both techniques are combined. I also used return_train_score=True so I could see how much the model was overfitting during the search.

```
grid_search = GridSearchCV(
    DecisionTreeClassifier(random_state=42),
    param_grid, cv=kfold,
    scoring='f1_weighted', n_jobs=-1,
    return_train_score=True
)
```

5.3 Overfitting Detection

I checked for overfitting in three ways for each model. First, I compared the mean train and test accuracy from K-Fold — a gap larger than 0.10 triggers a warning. Second, I compared the hold-out test accuracy to the train accuracy. Third, I plotted validation curves showing how train and test accuracy diverge as the key hyperparameter changes, so you can visually see the exact point where the model starts overfitting.

6. Model Results

6.1 Decision Tree

The Decision Tree was tested with a test size of 0.30. GridSearchCV picked entropy as the splitting criterion, max_depth of 7, min_samples_leaf of 4, and min_samples_split of 2.

K-Fold Cross Validation Results:

- Train Accuracy: 0.9275 ± 0.0049 — stable and consistent across all 5 folds.
- Test Accuracy: 0.8548 ± 0.0271 — solid generalisation with low variance.
- Overfit gap: 0.0727 — within the healthy range, no sign of overfitting.
- F1 Weighted: 0.8528 | F1 Macro: 0.8549

Hold-Out Test Set Results:

- Test Accuracy: 0.8621 | F1 Weighted: 0.8621 | F1 Macro: 0.8632
- ROC-AUC: 0.9555 — the model was very good at separating all three classes at the probability level.
- Grapes were classified perfectly (F1 = 1.00), which was not surprising given how distinct their weight range is.
- Apples and bananas scored 0.80 and 0.79 respectively, which is reasonable but shows there is still some confusion between the two.

```
Best hyperparameters: {'criterion': 'entropy', 'max_depth': 7, 'min_samples_leaf': 4, 'min_samples_split': 2}
Best CV F1 (validation) : 0.8658

========== Decision Tree — K-Fold CV Results (full dataset) ==========
  Train Accuracy  : 0.9275 ± 0.0049
  Test  Accuracy  : 0.8548 ± 0.0271
  Test  F1 (weighted): 0.8528 ± 0.0285
  Test  F1 (macro)   : 0.8549 ± 0.0280
  Overfit gap     : 0.0727  ✓ Healthy

========== Decision Tree — Hold-Out Test Set ==========
  Train Accuracy : 0.9333
  Test  Accuracy : 0.8621
  F1  (weighted) : 0.8621
  F1  (macro)    : 0.8632
  ROC-AUC (OvR)  : 0.9555

--- Classification Report ---
              precision    recall  f1-score   support

       apple       0.80      0.80      0.80        20
      banana       0.79      0.79      0.79        19
       grape       1.00      1.00      1.00        19

    accuracy                           0.86        58
   macro avg       0.86      0.86      0.86        58
weighted avg       0.86      0.86      0.86        58
```

Figure 1: Decision Tree — Evaluation Results (test_size=0.30)

6.2 Random Forest

The Random Forest was tested with a test size of 0.15. The best parameters were max_depth=10, max_features='sqrt', min_samples_leaf=2, min_samples_split=5, and n_estimators=200.

K-Fold Cross Validation Results:

- Train Accuracy: 0.9430 ± 0.0074 — slightly higher than Decision Tree.
- Test Accuracy: 0.8394 ± 0.0096 — very stable across folds, the lowest standard deviation of all models.
- Overfit gap: 0.1036 — just slightly above the 0.10 threshold, a mild sign of overfitting.
- F1 Weighted: 0.8380 | F1 Macro: 0.8404

Hold-Out Test Set Results:

- Test Accuracy: 0.8205 | F1 Weighted: 0.8205 | F1 Macro: 0.8272
- ROC-AUC: 0.9378 — still strong, though slightly lower than the Decision Tree.
- Grapes again scored perfectly (F1 = 1.00). Apples and bananas both scored 0.74.
- What surprised me here was that the Decision Tree actually outperformed the Random Forest, even though Random Forest is generally considered the stronger model. I think this is partly because the test size of 0.15 gave a smaller evaluation sample, and the dataset itself is small enough that a well-tuned single tree can do just as well.

```
Best hyperparameters: {'max_depth': 10, 'max_features': 'sqrt', 'min_samples_leaf': 2, 'min_samples_split': 5, 'n_estimators': 200}
Best CV F1 (validation) : 0.8823

========== Random Forest — K-Fold CV Results (full dataset) ==========
  Train Accuracy  : 0.9430 ± 0.0074
  Test  Accuracy  : 0.8394 ± 0.0096
  Test  F1 (weighted): 0.8380 ± 0.0095
  Test  F1 (macro)   : 0.8404 ± 0.0074
  Overfit gap     : 0.1036  ⚠ Possible overfit

========== Random Forest — Hold-Out Test Set ==========
  Train Accuracy : 0.9481
  Test  Accuracy : 0.8205
  F1  (weighted) : 0.8205
  F1  (macro)    : 0.8272
  ROC-AUC (OvR)  : 0.9378

--- Classification Report ---
              precision    recall  f1-score   support

       apple       0.77      0.71      0.74        14
      banana       0.71      0.77      0.74        13
       grape       1.00      1.00      1.00        12

    accuracy                           0.82        39
   macro avg       0.83      0.83      0.83        39
weighted avg       0.82      0.82      0.82        39
```

Figure 2: Random Forest — Evaluation Results (test_size=0.15)

6.3 Logistic Regression

The Logistic Regression was also tested with a test size of 0.15. GridSearchCV selected C=0.01, which means very strong regularization. This is already a sign that the model needed a lot of constraint to avoid poor performance — in other words, the data is simply not linearly separable.

K-Fold Cross Validation Results:

- Train Accuracy: 0.7306 ± 0.0194 — noticeably lower than both tree models.
- Test Accuracy: 0.7198 ± 0.0965 — the high variance here (±0.0965) shows the model is unstable across different data splits.
- Overfit gap: 0.0107 — practically no overfitting, but this is actually a sign of underfitting rather than a healthy model.
- F1 Weighted: 0.7088 | F1 Macro: 0.7134

Hold-Out Test Set Results:

- Test Accuracy: 0.6410 | F1 Weighted: 0.6124 | F1 Macro: 0.6171
- ROC-AUC: 0.8193 — still reasonable at the probability level, but the actual classifications were often wrong.
- Bananas were the hardest class with only 0.38 F1 and a recall of 0.31 — meaning the model missed more than two thirds of the bananas.
- My interpretation is that apples and bananas overlap too much in the current feature space for a linear boundary to work. The model simply cannot draw a straight line that cleanly separates them.

```
Best hyperparameters: {'model__C': 0.01, 'model__penalty': 'l2', 'model__solver': 'lbfgs'}
Best CV F1 (validation) : 0.7359

========== Logistic Regression — K-Fold CV Results (full dataset) ==========
  Train Accuracy  : 0.7306 ± 0.0194
  Test  Accuracy  : 0.7198 ± 0.0965
  Test  F1 (weighted): 0.7088 ± 0.1007
  Test  F1 (macro)   : 0.7134 ± 0.0967
  Overfit gap     : 0.0107  ✓ Healthy

========== Logistic Regression — Hold-Out Test Set ==========
  Train Accuracy : 0.7532
  Test  Accuracy : 0.6410
  F1  (weighted) : 0.6124
  F1  (macro)    : 0.6171
  ROC-AUC (OvR)  : 0.8193

--- Classification Report ---
              precision    recall  f1-score   support

       apple       0.64      0.64      0.64        14
      banana       0.50      0.31      0.38        13
       grape       0.71      1.00      0.83        12

    accuracy                           0.64        39
   macro avg       0.62      0.65      0.62        39
weighted avg       0.61      0.64      0.61        39
```

Figure 3: Logistic Regression — Evaluation Results (test_size=0.15)

7. Model Comparison

The table below shows all key metrics side by side across the three models.

Metric	Decision Tree	Random Forest	Logistic Regr.
Best Test Size	0.30	0.15	0.15
Best CV F1	0.8658	0.8823	0.7359
KFold Train Acc	0.9275 ± 0.0049	0.9430 ± 0.0074	0.7306 ± 0.0194
KFold Test Acc	0.8548 ± 0.0271	0.8394 ± 0.0096	0.7198 ± 0.0965
Overfit Gap	0.0727 ✓	0.1036 ⚠	0.0107 ✓
Hold-Out Test Acc	0.8621	0.8205	0.6410
F1 Weighted	0.8621	0.8205	0.6124
F1 Macro	0.8632	0.8272	0.6171
ROC-AUC (OvR)	0.9555	0.9378	0.8193
Grape F1	1.00	1.00	0.83
Apple F1	0.80	0.74	0.64
Banana F1	0.79	0.74	0.38

7.1 Overall Performance

The Decision Tree gave the best overall results with 86.21% test accuracy and the highest ROC-AUC of 0.9555. What I found surprising is that it outperformed Random Forest, which is generally considered the stronger model. I think this happened because the dataset is relatively small (193 records after cleaning), and with a small dataset a well-tuned single tree can sometimes match or even beat an ensemble. The larger test size of 0.30 also gave the Decision Tree a bigger and more representative evaluation set.

The Random Forest came in second with 82.05% test accuracy. Its biggest strength was stability — the K-Fold standard deviation was only ±0.0096, meaning its performance barely changed across different data splits.

Logistic Regression clearly struggled with this dataset, finishing at 64.10% test accuracy. In my opinion, the main reason is that apples and bananas are not linearly separable with the current three features. The very low C value of 0.01 selected by GridSearchCV confirms this — the model needed very strong regularization just to avoid performing even worse.

7.2 Per-Class Analysis

Grapes were easy to classify for every model. This makes sense — grapes are much lighter than the other two fruits, so weight alone is enough to identify them reliably.

Apples and bananas were the difficult cases. Their weight ranges overlap, and their colors and sizes can be similar. The tree models handled this better because they can create non-linear boundaries. Logistic Regression's banana recall of just 0.31 is particularly striking — it missed more than two thirds of the bananas.

7.3 Recommendation

I would recommend the Decision Tree as the primary model for this task. It gave the best accuracy, the highest ROC-AUC, and it is easy to interpret and explain. Random Forest is a solid backup if you need more stability. Logistic Regression in its current form is not a good fit for this problem.

8. Advantages and Disadvantages of Each Model

8.1 Decision Tree

Advantages:

- Easy to interpret — you can visualise the tree and explain each decision clearly.
- No feature scaling needed.
- Handles non-linear boundaries naturally.
- Fast to train.

Disadvantages:

- Prone to overfitting if the depth is not controlled.
- Sensitive to changes in training data — a small change can produce a completely different tree.

8.2 Random Forest

Advantages:

- Much more robust than a single tree — averaging many trees reduces the chance of overfitting.
- Provides feature importance scores, which is useful for understanding the data.
- Very stable results across different data splits.

Disadvantages:

- Harder to interpret — you cannot easily explain why a specific prediction was made.
- Slower to train because it builds many trees.
- Uses more memory.

8.3 Logistic Regression

Advantages:

- Very fast and simple.
- Easy to interpret — coefficients directly show each feature's contribution.
- Low risk of overfitting thanks to built-in regularization.

Disadvantages:

- Only works well when classes are linearly separable — which was not the case here.
- Requires feature scaling.
- Cannot capture non-linear relationships between features.

9. Future Work

Looking back at this project, there are several things I would do differently or add if I had more time.

First, I would run a proper correlation analysis before building the models. Looking at a heatmap of feature correlations would have helped me understand from the start which features actually matter and which ones are just noise. Based on the model results, weight is clearly the most important feature, but I only confirmed this after the fact by looking at the Random Forest feature importance plot. Doing this upfront could help guide better feature selection decisions.

Second, I would try Naive Bayes and Neural Networks as additional classifiers. Naive Bayes is known to work well on small datasets with categorical features, which matches this problem quite closely. A Neural Network, on the other hand, could potentially learn the non-linear boundaries between apples and bananas that Logistic Regression failed to capture. Adding these two models would give a much more complete picture of what works best for this type of data.

Third, I would do feature engineering before modelling. The three features we have — color, size, and weight — are not enough to cleanly separate apples and bananas. Adding new features like shape, texture, or a weight-to-size ratio could give the models more information to work with and likely improve classification accuracy on those two classes significantly.

Finally, collecting more data would help. 193 records is quite small for a machine learning project. A larger dataset would make the cross-validation results more reliable and give ensemble methods like Random Forest a better chance to show their true strength.

10. Conclusion

This project covered the full machine learning pipeline from raw data to evaluated models. The dataset had several quality issues that needed to be fixed before any model could be trained, including typos, digit contamination in text columns, and per-fruit weight outliers. All of these were handled systematically in the preprocessing cell.

I built three classification models — Decision Tree, Random Forest, and Logistic Regression — all tuned with GridSearchCV and Stratified K-Fold Cross Validation. The Decision Tree gave the best results overall with 86.21% test accuracy and a ROC-AUC of 0.9555. Random Forest was close behind at 82.05%, while Logistic Regression struggled at 64.10% due to the non-linear nature of the data.

One of the main things I learned from this project is that model selection really depends on the data. I went in expecting Random Forest to be the strongest model, and it turned out the simpler Decision Tree performed better on this specific dataset. I also found that Logistic Regression, despite being a standard baseline, is simply the wrong tool when the classes cannot be separated by a straight boundary.

For future improvement, adding more features, trying Naive Bayes and Neural Networks, and collecting more data would all likely push the accuracy higher — especially for the apple and banana classes, which remain the hardest to tell apart with the current feature set.